axiateam
RDT company

Anexo 12A

Vitest - Testing Determinista

Curso: TypeScript

75 min

Objetivos de Aprendizaje

- Configurar Vitest para tests deterministas (sin flaky tests)
- Configurar browser environments para testing en browser real
- Escribir tests unitarios con assertions confiables
- Escribir tests asíncronos robustos
- Utilizar mocks avanzados: vi.fn(), vi.mock(), vi.spyOn()
- Testear componentes React con Testing Library
- Medir coverage de tests
- Evitar falsos positivos: queries resilientes, cleanup apropiado

1. El Problema de los Tests No Deterministas

10 minutos

La Pesadilla de los Falsos Positivos

```
// TEST PROBLEMATICO
test('buscar tareas', () => {
  const service = new TaskService();
  service.addTask({ id: '1', title: 'Learn TypeScript', status: 'pending', priority: 'high' });

  // PROBLEMA: Este test pasa o falla aleatoriamente
  const results = service.searchTasks('typescript');

  expect(results.length).toBe(1); // Puede fallar por case sensitivity!
});
```

Causas comunes de falsos positivos:

- Tests que dependen de orden no garantizado (Set, Map)
- Tiempos (timers) no controlados
- Estado compartido entre tests
- Queries frágiles (getByText en lugar de getByRole)
- Mocks incompletos
- Falta de cleanup entre tests

Solución: Setup Determinista

- **Aislamiento:** `isolate: true`
- **Limpieza:** cleanup automático de mocks y estado
- **Control de tiempo:** `fakeTimers`
- **Entorno:** `jsdom` para browser simulado

2. Testing con Vitest

25 minutos

¿Qué es Vitest?

Framework de testing moderno desarrollado por el equipo de Vite.

- **Velocidad:** Usa el servidor de desarrollo de Vite
- **Compatibilidad:** API casi idéntica a Jest
- **ESM nativo:** Soporte completo para módulos ES
- **HMR:** Recarga los tests automáticamente
- **TypeScript:** Sin configuración adicional

Instalación

```
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom @testing-library/user-event
```

Configuración Determinista

```
// vite.config.ts
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: './src/test/setup.ts',
    isolate: true,
    pool: 'forks',
    poolOptions: {
      forks: { singleFork: true }
    },
    fakeTimers: {
      enableGlobally: ['setTimeout', 'setInterval', 'Date']
    },
    coverage: {
```

Setup de Testing Library

```
// src/test/setup.ts
import '@testing-library/jest-dom';
import { cleanup } from '@testing-library/react';
import { afterEach, vi } from 'vitest';

afterEach(() => {
  cleanup();
  vi.clearAllMocks();
  vi.useRealTimers();
});
```

Clave: `isolate: true` + cleanup = tests deterministas

Lab 1: Configurar Vitest

Objetivo: Configurar entorno de testing determinista

1. Instalar dependencias de testing
2. Crear archivo vitest.config.ts
3. Configurar jsdom con opciones de aislamiento
4. Crear setup.ts con cleanup automático
5. Agregar scripts al package.json

3. Tests con Mocks y Async

20 minutos

¿Por qué usar Mocks?

- **Aislamiento:** Testear sin depender de APIs externas
- **Velocidad:** Sin llamadas de red
- **Control:** Simular diferentes escenarios
- **Determinismo:** Resultados predecibles

vi.fn(), vi.mock(), vi.spyOn()

```
// vi.fn() - crear funciones mock
const mockCallback = vi.fn();
mockCallback.mockReturnValue(42);
mockCallback('arg'); // retorna 42

// vi.spyOn() - espiar métodos existentes
const api = { fetch: vi.fn() };
vi.spyOn(api, 'fetch').mockResolvedValue({ data: 'test' });

// vi.mock() - mockear módulos completos
vi.mock('./api', () => ({
  fetchTasks: vi.fn().mockResolvedValue([{ id: '1', title: 'Test' }])
}));
```

Tests Asíncronos

```
it('debe obtener tareas de forma asíncrona', async () => {
  const mockTasks = [{ id: '1', title: 'Learn TypeScript', status: 'pending' }];
  const fetchTasks = vi.fn().mockResolvedValue(mockTasks);

  const result = await fetchTasks();
  expect(result).toEqual(mockTasks);
});

it('debe manejar errores de API', async () => {
  const fetchConError = vi.fn().mockRejectedValue(new Error('Network error'));
  await expect(fetchConError()).rejects.toThrow('Network error');
});
```

Timers Fake

```
test('debounce funciona', () => {
  vi.useFakeTimers();

  const callback = vi.fn();
  const debounced = debounce(callback, 1000);

  debounced(); debounced(); debounced();

  expect(callback).not.toHaveBeenCalled();

  vi.advanceTimersByTime(1000);

  expect(callback).toHaveBeenCalledTimes(1); // Solo una vez

  vi.useRealTimers();
});
```

Lab 2: Tests con Mocking

Objetivo: Escribir tests que simulen llamadas a APIs

1. Crear el servicio TaskService
2. Escribir tests unitarios con mocking
3. Ejecutar los tests y verificar coverage

4. Testing de Componentes React

15 minutos

¿Qué es Testing Library?

Librería que prioriza probar componentes como el usuario real los usa.

- **Filosofía:** "El software más mantenible prueba el comportamiento"
- **Accesibilidad:** Queries priorizan ARIA
- **User-centric:** Simula interacción real

Jerarquía de Queries

De más a menos preferible:

1. `getByRole` - El más accesible (usa ARIA)
2. `getByLabelText` - Para formularios
3. `getByPlaceholderText` - Si no hay label
4. `getByText` - Solo elementos no interactivos
5. `getByTestId` - Último recurso

Queries en la Práctica

```
// ❌ Evitar - frágil
const button = screen.getByText('Submit');

// ✅ Preferir - semántico
const button = screen.getByRole('button', { name: /submit/i });

// ✅ Para formularios
const input = screen.getByLabelText(/email/i);

// ✅ Último recurso
const element = screen.getByTestId('unique-id');
```

Test de Componente

```
describe('TaskCard', () => {
  const mockTask = { id: '1', title: 'Learn TypeScript', status: 'pending' as const };

  it('debe renderizar el título', () => {
    render(<TaskCard task={mockTask} />);
    expect(screen.getByText('Learn TypeScript')).toBeInTheDocument();
  });

  it('debe llamar a onComplete al hacer clic', () => {
    const onComplete = vi.fn();
    render(<TaskCard task={mockTask} onComplete={onComplete} />);
    fireEvent.click(screen.getByRole('button', { name: /completar/i }));
    expect(onComplete).toHaveBeenCalledWith('1');
  });
});
```

Lab 3: Testear Componentes

Objetivo: Testear componentes React del sistema de tareas

1. Testear TaskCard con diferentes estados
2. Testear formulario de tareas
3. Testear contexto de tareas

5. TypeScript Avanzado en Tests

10 minutos

¿Por qué tipar los Mocks?

- **Error en compilación:** Si la firma cambia, el test falla antes
- **Autocompletado:** El IDE sugiere métodos correctos
- **Documentación:** El tipo explica qué espera el mock
- **Refactoring seguro:** Los tests se actualizan automáticamente

Tipado de Mocks en Vitest

```
// Tipado de funciones con vi.fn<() => Tipo>()
const mockFetchTasks = vi.fn<() => Promise<Task[]>>();

// Tipado de objetos con vi.mocked<T>()
import { TaskService } from './services/taskService';
const mockedService = vi.mocked(TaskService, true);

// Tipado de espías
const spyOnSearch = vi.spyOn(service, 'searchTasks')
  .mockReturnValue([taskMock]);

// Type assertion en retornos
const mockFetch = vi.fn<() => Promise<{ data: Task[] }>>();
mockFetch.mockResolvedValue({ data: [taskMock] });
```

Lab 4: Mocks Fuertemenete Tipados

Objetivo: Escribir tests con mocks completamente tipados

1. Definir interfaces para tipos de retorno
2. Crear mocks con vi.fn<()>()
3. Utilizar vi.mocked<T>() para objetos

6. Testing de Custom Hooks

10 minutos

¿Qué es renderHook?

Utilidad de Testing Library para probar custom hooks de forma aislada.

- **Aislamiento:** Testea el hook sin componente
- **Estado verificable:** Accede a result.current
- **Acciones:** Ejecuta funciones del hook
- **Wrapper:** Soporta contextos de React

API de renderHook

```
import { renderHook, act } from '@testing-library/react';
import { useTasks } from './hooks/useTasks';

test('debe retornar lista de tareas', async () => {
  const { result } = renderHook(() => useTasks());

  // Estado inicial
  expect(result.current.tasks).toEqual([]);
  expect(result.current.loading).toBe(true);

  // Esperar a que cargue
  await act(async () => {
    await vi.waitFor(() => expect(result.current.loading).toBe(false));
  });

  // Verificar datos
  expect(result.current.tasks.length).toBeGreaterThan(0);
});
```

renderHook con Context

```
// Con providers (Context)
const wrapper = ({ children }: { children: React.ReactNode }) => (
  <TaskProvider>{children}</TaskProvider>
);

const { result } = renderHook(() => useTasks(), { wrapper });

// Acceder al estado
console.log(result.current.tasks);
console.log(result.current.loading);
console.log(result.current.error);
```

Lab 5: Testear useTasks

Objetivo: Escribir tests completos para custom hook

1. Crear el hook useTasks
2. Testear estado inicial (loading = true)
3. Testear carga exitosa
4. Testear manejo de errores

7. Tests Parametrizados

10 minutos

¿Qué son los Tests Parametrizados?

Permiten ejecutar el mismo test con diferentes valores

- Reduce código duplicado
- Facilita agregar nuevos casos
- Mejor mantenimiento

describe.each

```
describe.each([
  { input: 'typescript', expected: 1 },
  { input: 'javascript', expected: 0 },
  { input: 'TS', expected: 1 },
])('searchTasks', ({ input, expected }) => {
  test(\`debe encontrar \${input}\`, () => {
    const resultado = service.searchTasks(input);
    expect(resultado.length).toBe(expected);
  });
});
```

test.each

```
test.each([
  ['low', true],
  ['medium', true],
  ['high', true],
  ['invalid', false],
])('validarPriority: %s', (priority, esValido) => {
  expect(validatePriority(priority)).toBe(esValido);
});
```

Lab 6: Tests Parametrizados

Objetivo: Escribir tests para validaciones de formulario

1. Crear función de validación
2. Escribir tests parametrizados
3. Casos: válido, sin título, título corto, sin prioridad

8. Ejercicio Integrador

Coverage mayor al 80%

Objetivo

Implementar tests completos para el sistema de biblioteca (Caso 2r)

- Escribir tests para todos los métodos del servicio
- Lograr al menos 80% de coverage
- Testear componentes principales (BookCard, BookList, LoanForm)

Pasos

1. Ejecutar `npm run test:coverage` para ver coverage actual
2. Identificar funciones sin tests
3. Agregar los casos de prueba faltantes
4. Verificar que se alcanza el 80% de coverage

Resumen

Puntos Clave

- Vitest = Jest + Vite = velocidad y compatibilidad
- `isolate: true` = tests limpios y deterministas
- `vi.useFakeTimers()` = control de tiempo
- `vi.fn()` = crear mocks
- `vi.spyOn()` = espiar métodos
- Queries: getByRole → getByLabelText → getByTestId
- setup.ts = configuración global con cleanup
- renderHook = testear custom hooks
- Tests parametrizados = evitar duplicación
- Mocks fuertemente tipados = seguridad en refactoring

Referencias

Recursos Adicionales

- Vitest Documentation
- Vitest API
- Vitest Configuration
- Vitest Coverage
- React Testing Library
- Query Priority
- Mock Functions
- Vi Mock
- Async Testing

Gracias!

Anexo 12A Completado

Siguiente: Anexo 12B - Playwright